

Software Supply Chain — A Plain-Language Briefing

Date: 2026-08-11

Audience: anyone who needs to understand and fund this. No technical background assumed.

Companion document: the detailed version with all the measurements is `npm-supply-chain-exposure-report.md`. Everything here comes from there. Nothing here is rounded up or dramatized.

Start here: this is not a report about something that happened to us

We went looking for evidence that an attack had reached us. We did not find any. We checked what our software is built from, what our automated systems have been doing, and what our laptops and servers have been running. Nothing showed a break-in.

This report is about the doors, not about a burglary. We looked at how hard it would be for someone to get in if they tried tomorrow, and the honest answer is: easier than it should be. That is a fixable problem, and most of it is fixable with configuration changes rather than purchases.

What are we talking about?

Almost no modern software is written from scratch. When our teams build something, they assemble it out of thousands of small, free, ready-made components published by strangers on the internet — the same way a manufacturer buys screws, motors and wiring rather than making them. There is a public catalog these components come from, and our automated build systems download from it constantly, without a person watching.

This is normal and it is how the entire industry works. It is also, increasingly, how the industry gets attacked.

The attack is simple. Someone takes over the account of a person who publishes a popular component. They publish a new version of it with a small piece of hostile code added. Every company in the world that builds software that week picks it up automatically. Nobody downloaded anything suspicious; nobody clicked a bad link; nobody made a mistake. The component was trusted last week, so it was trusted this week.

That is what has been happening across the industry through 2026, on a large scale. This briefing is about how well we would hold up.

What is the problem?

Five things. They are separate problems, but they line up into a single chain, and that is what makes them serious together.

1. We hand over the whole keyring

Our automated build systems hold the passwords and keys needed to do their jobs — to reach our cloud accounts, our databases, our deployment systems. Reasonable.

The problem is how those keys are passed around. When one of our shared build components starts up, we do not hand it the one key it needs. **We hand it every key we have.**

Think of a contractor arriving to fix a supply closet, and being given the master key to every door in the building, including the ones they have no business opening.

How widespread: this happens in **1,924** places across our build systems. Almost all of them — **1,904** — go through a single shared component. That is bad news and good news at the same time: it is a lot of exposure, but it is concentrated, so fixing a handful of things fixes nearly all of it.

Separately, **46** of our shared building blocks pass the whole keyring onward to whatever called them, reaching **1,823** projects. **37** of those go on to deploy to production.

2. We run supplier code on sight

When one of these free components is downloaded, it is allowed to bring along an instruction that says, in effect, "*before you use me, run this.*" Our build systems obey that instruction automatically, before anyone or anything has looked at it.

This is the front door of the attack. It is where these attacks begin, every time. And there is a single setting that says "download the component, but do not run its instructions." Almost nothing of ours sets it.

How widespread: of the projects that download components during an automated build, only **6** refuse to run the instructions. **94** projects run them.

3. Nine of our automated processes are calling a vacant address

Our automated processes routinely say "go and get your instructions from over there." Nine of them are pointing at an address inside our own systems where nothing lives any more — the instructions they name were deleted a while ago.

Right now that just means those nine processes are broken. The real issue is that **the address is vacant, not sealed.** Anyone with ordinary access inside the company can move into that vacant address and start supplying instructions — and, because of problem 1, whatever instructions they supply get handed the keyring.

The addresses are not secret. Anybody can see which ones our processes are calling, simply by reading our own project files.

This one deserves particular attention because it does not require an outside attacker or a poisoned component at all. It is reachable by anyone already inside.

4. We track moving targets rather than fixed ones

When our builds reference an outside component, they usually say "give me the current version" rather than "give me exactly this one, the one we checked." That means the thing that runs in our systems tomorrow can be different from the thing that ran today, without anybody changing anything or approving anything.

How widespread: about **one in ten** of our references name an exact, fixed version. The other nine in ten move.

5. Keys that never expire, and no single place to say no

Two structural gaps sit underneath the four problems above:

- The keys our systems hold are **long-lived**. If one is taken, it stays useful to the thief for months, from anywhere in the world.
- There is **no single place** where we can say "block that component" and have it apply everywhere at once. Every project fetches independently, straight from the public catalog.

What happens if we do not fix this?

Not "the company falls over." Something more specific and more expensive.

A very small intrusion becomes a very large one. With the keyring problem, an attacker who gets even a second of execution inside one build does not get one credential — they get all of them, including the ones that reach production. There is no small version of this incident.

The clean-up is the real cost. Once you cannot prove which keys were taken, you have to assume all of them were. That means rotating every credential across the estate, re-deploying systems, and doing it under time pressure while normal work stops. That is a multi-week, all-hands exercise, and it is triggered by an event that might have lasted a few seconds.

We would struggle to answer the question everyone asks. After an incident, the first questions are "what did they take, and where did they go?" Some of our records do not capture enough detail to answer that precisely. We would be reasoning from what was possible rather than from what was recorded — which lengthens the response and weakens what we can tell customers, auditors and regulators.

And the odds are not static. These attacks are getting more frequent, more automated, and better at spreading on their own. Our exposure is not currently going down by itself.

Is there a benefit beyond security?

Yes, and this is worth making explicit, because several of these fixes pay for themselves even if nobody ever attacks us.

Builds stop changing underneath us. Fixing problem 4 means the software we build tomorrow is assembled from exactly the same parts as today. A large share of "it worked yesterday and today it does not" failures come from exactly this — an outside component changed without warning. Naming fixed versions removes a recurring category of unexplained breakage and the engineering hours spent chasing it.

Nine broken things get fixed. The nine vacant addresses in problem 3 are not only a risk; they are nine automated processes that currently do not work.

We find out what we actually use. Routing everything through one internal supply point gives us, for the first time, a reliable list of every outside component in our products. Today that question takes a project to answer. Afterwards it is a report. That list is the thing customers and auditors increasingly ask for by name.

We become less dependent on someone else's uptime. When the public catalog has an outage — which happens — every one of our builds stops. An internal supply point holds copies, so our work continues.

Incidents get shorter. Short-lived keys mean that after an incident, most of what an attacker took has already expired. That converts "rotate everything, verify everything" into a much smaller, calmer piece of work.

One place to act. The next time an advisory names a bad component, the response is one change in one place instead of a hunt across hundreds of projects.

What is involved in fixing it?

Nothing here requires buying a product. All of it is changing settings and habits in systems we already own and already pay for.

Now — a few weeks, and mostly small changes

A. Close the nine open doors. Repoint the nine broken processes at somewhere real, and then reserve the vacant addresses so nobody else can move in. Both halves matter; doing only the first

leaves the door shut but unlocked. *Hours of work. This is the sharpest risk we found and the least effort to fix — it should start first.*

B. Stop running supplier code on sight. Turn on the setting that downloads a component but refuses to run the instructions it brings with it. *One line per project, across 94 projects. Expect a small number of components that genuinely need to run something — those get reviewed and recorded as named exceptions rather than waved through. Cheapest protection in this report.*

C. Stop handing over the whole keyring. Change our shared build components so each job receives only the credential it needs. *A few weeks. It sounds enormous and it is not, because the exposure is concentrated: a small number of shared components account for nearly all of it, so we fix those rather than thousands of projects.*

Next — a quarter each, in parallel with normal work

D. Badges instead of keys. Replace stored passwords with short-lived passes issued to each job as it starts and expiring minutes later, so there is nothing durable left to steal. *We already own this capability and are barely using it — it is configuration, not a purchase. It is also what makes fix C permanent rather than a setting somebody can quietly undo later.*

E. One front door for outside software. Route everything through a single managed internal supply point, with two features switched on: a block list that applies everywhere at once, and a short waiting period before a brand-new version is allowed into our builds.

This one has an unusually good starting position: **we already own the front door and almost nothing goes through it.** The internal supply point exists and is already connected to the public catalog. Our Microsoft-ecosystem components already flow through it — 523 of them. The components in question here: zero. The mechanism is proven inside our own walls; it was simply never pointed at this category of software.

Why the waiting period works, in numbers rather than theory: of the 2,208 bad versions published in this campaign, **2,097 had already been pulled from the public catalog** by the time we checked — around 95%. The catalog cleans itself up, but only after a few days, and our builds are currently fast enough to catch them in that gap. Making a new version sit for a few days before any build may use it means most of these are gone before we could ever have touched them. The remaining ~5% are why the block list is needed as well.

What we are not claiming

Plain about the limits, because a briefing that overstates its confidence is worth less than one that does not:

- **We did not check everything.** We examined the main version of each project, not every variation. In 193 projects, the download step happens somewhere we did not read, so we do not know whether they are affected. **Not checked is not the same as clean**, and we have not counted it as clean.
- **Personal work machines are largely outside this.** Most of the real-world victims of these campaigns were hit on a developer's own laptop, not on a company build server. Our visibility there is thinner than it is on servers.
- **One internal supply point could not be read** for permission reasons. A request to fix that is filed and named in the detailed report.
- **"No evidence of a break-in" is not the same as "no break-in occurred."** It means every place we knew to look, and could look, was clean. We have written down exactly where we could not look, so nobody mistakes a gap for a green light.

The one-page version

We build our products partly out of free components downloaded automatically from the internet. Attackers have started poisoning those components at the source, and it is working across the industry. We checked, and it has not reached us.

But if it did, three things would turn a small event into a large one: we let downloaded components run their own instructions automatically, we hand our build systems every password we have instead of the one they need, and nine of our automated processes are calling an address anyone inside the company could claim.

Fixing the first three costs weeks, not quarters, and requires no purchase. Two further changes — short-lived passes instead of stored passwords, and a single inspected entry point for outside software — take a quarter each and stop this from being work we repeat after the next campaign. Several of them also fix problems we already have: broken automation, unexplained build failures, and no reliable inventory of what our software is made of.

A few terms, if you want them

Term	What it means here
Component / package	A ready-made piece of software written by someone else that our products are assembled from. We use thousands.
The public catalog	The free, public, largely unpoliced library these components are published to and downloaded from.
Build system	The automated process that assembles our software and sends it to production, without a person in the loop.
Credential / key	A password or token that lets a system prove who it is and get access to something.
Supply chain attack	An attack that reaches you through something you trusted and installed, rather than through your own front door.
